

Beyond Pass-by-Pass Optimization: Intent-Driven IR Optimization with Large Language Models

超越逐遍优化: 使用大语言模型的意图驱动的中表示优化

Lei Qiu¹² Zi Yang¹³ Fang Lyu¹ Ming Zhong⁴ Huimin Cui¹² Xiaobing Feng¹²

雷邱¹² 紫阳¹³ 吕方¹ 钟鸣⁴ 崔慧敏¹² 冯小兵¹²

Abstract

摘要

Modern compilers optimize programs through a sequence of modular passes over intermediate representations (IR). While this pass-by-pass paradigm offers engineering benefits, it suffers from a pass coordination problem: locally beneficial transformations may block more profitable optimizations in later stages. This limitation stems from the lack of an explicit notion of optimization intent, defined as a holistic strategy for coordinating multiple transformations toward a global performance objective. Recent LLM-based approaches formulate IR optimization as an end-to-end generation task, thereby avoiding the traditional pass-by-pass structure. However, optimization intent remains implicit in these methods, forcing models to jointly infer optimization strategy and generate low-level transformations, which limits both correctness and performance. We propose IntOpt, the first intent-driven IR optimizer that explicitly separates high-level optimization intent from low-level analysis and transformation. IntOpt organizes IR optimization into three stages: intent formulation, intent refinement, and intent realization, enabling globally coordinated transformations. Experiments show that IntOpt achieves 90.5% verified correctness and 2.660 $\times$ average speedup on 200-program test set, outperforming state-of-the-art LLM-based optimizers in both correctness and performance, and surpassing modern compiler with the -O3 option on 37 benchmarks with speedups of up to 272.60 $\times$.

现代编译器通过对中间表示 (IR) 进行一系列模块化遍历来优化程序。虽然这种逐遍范式带来了工程上的好处, 但它存在遍协调问题: 局部有益的转换可能会在后期阶段阻碍更有利可图的优化。这种限制源于缺乏明确的优化意图概念, 优化意图被定义为一种协调多个转换以实现全局性能目标的整体策略。最近基于大语言模型的方法将 IR 优化表述为端到端生成任务, 从而避免了传统的逐遍结构。然而, 优化意图在这些方法中仍然是隐含的, 迫使模型联合推断优化策略并生成低级转换, 这限制了正确性和性能。我们提出了 IntOpt, 这是第一个意图驱动的 IR 优化器, 它明确地将高级优化意图与低级分析和转换分开。IntOpt 将 IR 优化组织为三个阶段: 意图制定、意图细化和意图实现, 实现全局协调的转换。实验表明, IntOpt 在 200 个程序的测试集上实现了 90.5% 的验证正确性和 2.660 倍的平均加速比, 在正确性和性能方面均优于基于大语言模型的现有优化器, 并且在 37 个基准测试中比使用 -O3 选项的现代编译器实现了高达 272.60 倍的加速比。

1. Introduction

1. 引言

Modern compilers leverage intermediate representations (IR) to enable compilation that is independent of both programming languages and hardware architectures (Aho et al., 2006; Lattner & Aive, 2004). By optimizing at the IR level, which abstracts away language- and hardware-specific details, compilers can improve the program execution efficiency (referred to as "performance") across diverse programming languages, target architectures, and application domains, ranging from traditional scientific computing (Moses et al., 2023; Bezanson et al., 2017; Özen et al., 2018) to modern AI workloads with specialized tensor operations (Lattner et al., 2021; Chen et al., 2018; Sabne, 2020). Consequently, designing effective IR optimization strategies has remained significant in compiler construction.

现代编译器利用中间表示 (IR) 来实现独立于编程语言和硬件架构的编译 (Aho 等人, 2006 年; Lattner 和 Aive, 2004 年)。通过在 IR 级别进行优化, 即抽象出语言和硬件特定细节, 编译器可以提高各种编程语言、目标架构和应用领域的程序执行效率 (称为“性能”), 范围从传统科学计算 (Moses 等人, 2023 年; Bezanson 等人, 2017 年; Özen 等人, 2018 年) 到具有专门张量操作的现代人工智能工作负载 (Lattner 等人, 2021 年; Chen 等人, 2018 年; Sabne, 2020 年)。因此, 设计有效的 IR 优化策略在编译器构建中一直很重要。

However, the effectiveness of IR-level optimization is fundamentally constrained by how modern compilers organize optimization. In practice, modern compilers decompose IR optimization into a sequence of passes, each implemented as a modular unit that targets a specific optimization, such as loop unrolling or constant propagation (Lattner & Aive, 2004; Stanier & Watson, 2013). These passes make decisions by analyzing optimization-related information derived from the IR (hereafter referred to as "analysis") to assess whether a transformation will be beneficial. Although this pass-by-pass paradigm offers clear engineering benefits in terms of modularity and maintainability, it inherently suffers from a pass coordination problem: optimizations that are locally beneficial in early passes can inadvertently restrict or even prevent more profitable optimizations in later stages.

然而, IR 级优化的有效性从根本上受到现代编译器组织优化方式的限制。在实践中, 现代编译器将 IR 优化分解为一系列遍, 每个遍都作为针对特定优化 (如循环展开或常量传播) 的模块化单元实现 (Lattner 和 Aive, 2004 年; Stanier 和 Watson, 2013 年)。这些遍通过分析从 IR 派生的与优化相关的信息 (以下称为“分析”) 来做出决策, 以评估转换是否有益。尽管这种逐遍范式在模块化和可维护性方面提供了明显的工程优势, 但它固有地存在遍协调问题: 早期遍中局部有益的优化可能会无意中限制甚至阻止后期阶段更有利可图的优化。

As illustrated in Fig. 1(b), when the compiler optimizes the IR corresponding to the source code in Fig. 1(a), the LICM-Pass (loop-invariant code motion) hoists $D[i]$ from L_3 to L'_2 to eliminate redundant accesses in the inner loop. While this transformation is locally beneficial, it prevents the LoopInterchangePass from swapping the i and j loops to improve memory locality. More critically, because LoopInterchangePass cannot be applied, the subsequent LoopVectorizePass also fails to vectorize L'_4 . As a result, the innermost loop retains a column-major access pattern for $A[j][i]$ and $B[j][i]$, which inhibits effective SIMD utilization and ultimately leads to degraded performance.

如图 1(b) 所示, 当编译器优化图 1(a) 中对应源代码的 IR 时, LICM-Pass(循环不变代码外提) 将 $D[i]$ 从 L_3 提升到 L_2' 以消除内循环中的冗余访问。虽然这种转换在局部是有益的, 但它阻止了 Loop-InterchangePass 交换 i 和 j 循环以改善内存局部性。更关键的是, 由于无法应用 LoopInterchangePass, 后续的 LoopVector-izePass 也无法对 L_4' 进行向量化。结果, 最内层循环对 $A[j][i]$ 和 $B[j][i]$ 保留了按列优先的访问模式, 这抑制了有效的 SIMD 利用并最终导致性能下降。

This behavior arises from the lack of an explicit notion of optimization intent, which we define as a holistic strategy for coordinating multiple transformations toward a global performance objective. In compilers such as GCC (GCC, 2026) and LLVM (LLVM, 2026a), this intent is implicit and fragmented across individual optimization passes. As shown in Fig. 1(b), LICMPass reduces memory operations, Loop-InterchangePass improves memory locality, and LoopVec-torizePass targets SIMD optimization. However, without explicit global coordination, individual passes cannot anticipate how their transformations may enable or inhibit downstream optimization opportunities (Kulkarni et al., 2006; Pallister et al., 2015).

这种行为源于缺乏明确的优化意图概念, 我们将其定义为一种协调多个转换以实现全局性能目标的整体策略。在诸如 GCC(GCC, 2026) 和 LLVM(LLVM, 2026a) 等编译器中, 这种意图是隐含的并且分散在各个优化遍中。如图 1(b) 所示, LICMPass 减少内存操作, Loop-InterchangePass 改善内存局部性, LoopVec-torizePass 针对 SIMD 优化。然而, 由于缺乏明确的全局协调, 各个遍无法预测其转换如何可能启用或抑制下游优化机会 (Kulkarni 等人, 2006 年; Pallister 等人, 2015 年)。

¹ SKLP, ICT, CAS, China ² UCAS, China ³ Jiangnan University, China ⁴CUHK, China. Correspondence to: Fang Lyu <flv@ict.ac.cn>, Huimin Cui <cuihm@ict.ac.cn>.

¹ 中国科学院软件研究所, 中国 ² 中国科学院大学, 中国 ³ 江南大学, 中国 ⁴CUHK, 中国。通信作者: 吕方 <flv@ict.ac.cn>, 崔慧敏 <cuihm@ict.ac.cn>。

Preprint. February 24, 2026.

预印本。2026 年 2 月 24 日。

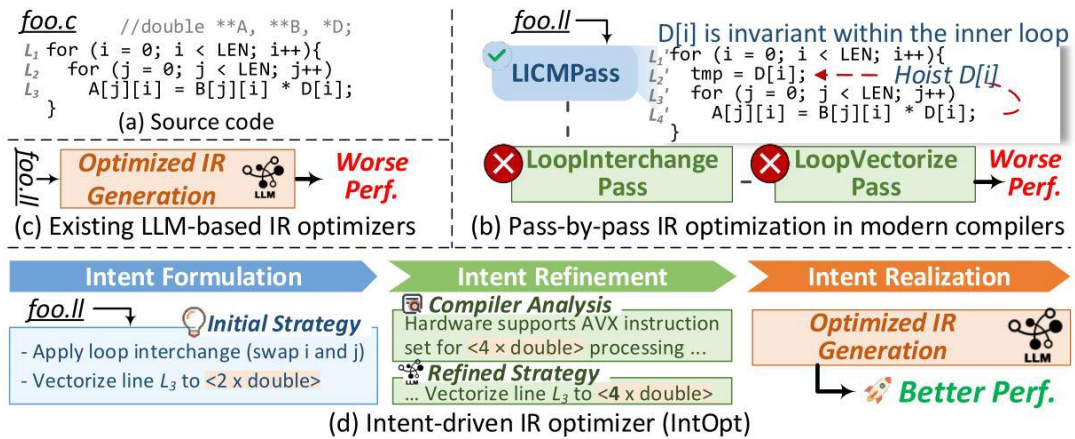


Figure 1. Comparison between existing IR optimization approaches and our INTOPT.

图 1. 现有 IR 优化方法与我们的 INTOPT 之间的比较。

Recent advances in large language models (LLMs) (OpenAI, 2025; Anthropic, 2025; Daya et al., 2025) offer a promising way to address this limitation due to their strong program understanding and semantic reasoning capabilities. Prior LLM-based IR optimization methods (Cummins et al., 2025; Yang et al., 2025) formulate IR optimization as an end-to-end generation task that directly maps unoptimized IR to optimized IR, as shown in Fig. 1(c), avoiding the traditional pass-by-pass structure in compilers. However, this reformulation does not fundamentally resolve the core issue: optimization intent remains implicit, now hidden within a black-box LLM generation process rather than fragmented across individual passes in compilers. Consequently, LLMs are forced to simultaneously infer optimization intent and generate precise transformations, which makes it challenging to guarantee both semantic correctness and consistent performance improvement. Our experiments in Sec. 4.2 empirically reveal substantial room for improvement.

大语言模型 (LLMs)(OpenAI, 2025; Anthropic, 2025; Daya 等人, 2025) 的最新进展提供了一种很有前景的方法来解决这一限制, 因为它们具有强大的程序理解和语义推理能力。先前基于 LLM 的 IR 优化方法 (Cummins 等人, 2025; Yang 等人, 2025) 将 IR 优化表述为一个端到端的生成任务, 直接将未优化的 IR 映射到优化后的 IR, 如图 1(c) 所示, 避免了编译器中传统的逐遍结构。然而, 这种重新表述并没有从根本上解决核心问题: 优化意图仍然是隐含的, 现在隐藏在一个黑盒 LLM 生成过程中, 而不是分散在编译器的各个单独遍中。因此, LLMs 被迫同时推断优化意图并生成精确的转换, 这使得保证语义正确性和一致的性能提升具有挑战性。我们在第 4.2 节中的实验从经验上揭示了有很大的改进空间。

In this paper, we propose INTOPT, an Intent-driven IR Optimizer that reformulates the modern compiler optimization paradigm by decoupling high-level optimization intent from low-level analysis and transformation, thereby enabling globally coordinated transformations. INTOPT achieves this goal by deconstructing the IR optimization process into three stages: intent formulation, intent refinement, and intent realization. This pipeline mirrors how expert compiler engineers operate: first reasoning about optimization intent at a global level, then grounding it using program analysis to assess feasibility, and finally realizing it through concrete transformations (Muchnick, 1998).

在本文中, 我们提出了 INTOPT, 一种意图驱动的 IR 优化器, 它通过将高级优化意图与低级分析和转换解耦, 重新构建了现代编译器优化范式从而实现全局协调的转换。INTOPT 通过将 IR 优化过程解构为三个阶段来实现这一目标: 意图制定、意图细化和意图实现。这个流程反映了专家编译器工程师的操作方式: 首先在全局层面推理优化意图, 然后使用程序分析来评估其可行性, 最后通过具体转换来实现它 (Muchnick, 1998)。

Our key insight is that, although existing compilers encode optimization intent implicitly within hand-crafted passes, these intents capture valuable domain knowledge that LLMs can learn and make explicit. Building on this, INTOPT first uses an LLM specialized in compiler optimization to infer an initial optimization strategy for a given program, representing a structured form of optimization intent. This strategy is then grounded and refined using compiler analysis. For example, in Fig. 1(d), an initial strategy of vectorization with $\langle 2 \times \text{double} \rangle$ is refined to $\langle 4 \times \text{double} \rangle$ based on hardware-aware analysis. The refined strategy subsequently guides optimized IR generation, ensuring that transformations are globally coordinated. During the intent refinement and realization stage, INTOPT leverages GPT-5 (OpenAI, 2025) and its pre-training knowl-

edge to explore optimization opportunities beyond those captured by existing compiler passes, as shown in Sec. 4.4.

我们的关键见解是，尽管现有编译器在手工编写的遍中隐含地编码了优化意图，但这些意图包含了 LLMs 可以学习并明确的有价值的领域知识。基于此，INTOPT 首先使用专门用于编译器优化的 LLM 为给定程序推断初始优化策略，这代表了一种结构化的优化意图形式。然后使用编译器分析对该策略进行落地和细化。例如，在图 1(d) 中，基于硬件感知分析，将初始的 $\langle 2 \times \text{double} \rangle$ 向量化策略细化为 $\langle 4 \times \text{double} \rangle$ 。细化后的策略随后指导优化后的 IR 生成，确保转换是全局协调的。在意图细化和实现阶段，INTOPT 利用 GPT - 5 (OpenAI, 2025) 及其预训练知识来探索现有编译器未涵盖的优化机会，如第 4.4 节所示。

We evaluate INTOPT on 200 LLVM IR programs and achieve significant improvements over both end-to-end LLM-based optimizers and traditional compiler. Our results highlight the importance of making optimization intent explicit for globally coordinated and semantically grounded IR optimization. We hope our findings will facilitate more research on intent-driven compiler optimization in this field.

我们在 200 个 LLVM IR 程序上评估了 INTOPT，并在基于 LLM 的端到端优化器和传统编译器上都取得了显著改进。我们的结果突出了明确优化意图对于全局协调和语义基础的 IR 优化的重要性。我们希望我们的发现将促进该领域对意图驱动的编译器优化的更多研究。

To sum up, our main contribution includes:

综上所述，我们的主要贡献包括：

- Intent-driven IR optimization framework. We propose the first intent-driven IR optimization framework that decouples optimization intent from concrete IR transformation, enabling more effective and reliable optimizations.

- 意图驱动的 IR 优化框架。我们提出了第一个意图驱动的 IR 优化框架，该框架将优化意图与具体的 IR 转换解耦，实现更有效和可靠的优化。

- Compiler-knowledge-grounded IR optimization. We develop a novel approach that integrates structured compiler knowledge to guide LLMs in IR optimization.

- 基于编译器知识的 IR 优化。我们开发了一种新颖的方法，将结构化的编译器知识集成到 IR 优化中以指导 LLMs。

- Comprehensive evaluation. INTOPT achieves 90.5% verified correctness and $2.660\times$ average speedup on 200- program test set, outperforming state-of-the-art LLM-based optimizers by 7.0%-31.5% in correctness and up to $1.603\times$ in performance, while also surpassing LLVM -03 on 37 benchmark programs with speedups up to $272.60\times$.

- 综合评估。INTOPT 在 200 个程序测试集上实现了 90.5% 的验证正确性和 $2.660\times$ 平均加速比，在正确性方面比基于 LLM 的最先进优化器高出 7.0% - 31.5%，在性能方面最高可达 $1.603\times$ ，同时在 37 个基准程序上也超过了 LLVM - 03，加速比高达 $272.60\times$ 。

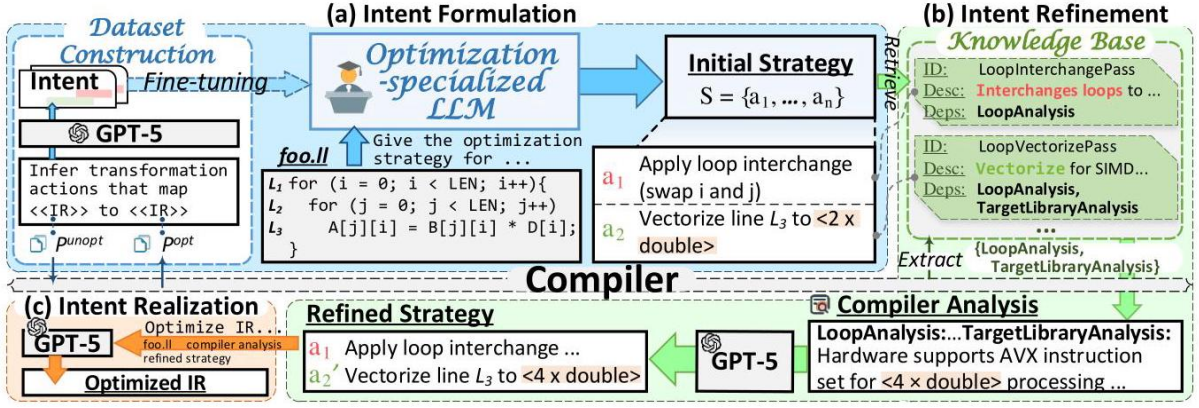


Figure 2. Overview of INTOPT.

图 2. INTOPT 概述。

2. Related Works

2. 相关工作

Traditional IR Optimization in Compilers. Modern compilers such as LLVM (LLVM, 2026a) and GCC (GCC, 2026) perform IR optimization through a fixed sequence of passes (e.g., -O3), where each pass applies a specific transformation guided by a predefined set of compiler analysis and hand-crafted heuristics (Lattner & Adev, 2004). To provide flexibility, compilers expose numerous configuration options that allow users to enable or disable specific optimizations or adjust heuristic parameters (Muchnick, 1998). However, interactions among optimization passes are often complex and difficult to predict (Kulkarni et al., 2006; Pal-lister et al., 2015), making it challenging to reason about or control global optimization strategies for a given program.

编译器中的传统 IR 优化。现代编译器如 LLVM(LLVM, 2026a) 和 GCC(GCC, 2026) 通过固定的遍序列 (例如 -O3) 执行 IR 优化, 其中每个遍应用由预定义的一组编译器分析和手工编写的启发式方法指导的特定转换 (Lattner & Adev, 2004)。为了提供灵活性, 编译器公开了许多配置选项, 允许用户启用或禁用特定优化或调整启发式参数 (Muchnick, 1998)。然而, 优化遍之间的交互通常很复杂且难以预测 (Kulkarni 等人, 2006; Pal - lister 等人, 2015), 这使得推理或控制给定程序的全局优化策略具有挑战性。

Autotuning for Compiler Optimization. Building on the pass-by-pass optimization infrastructure, a line of work (Liang et al., 2023; Ansel et al., 2014; Chen et al., 2021; Zhu et al., 2024; Park et al., 2022; Pan et al., 2025) has explored autotuning approaches that search over compiler options, optimization flags, or pass configurations to improve performance. While these methods can yield performance gains, it suffers from several inherent limitations. First, the search process is time-consuming, as it requires repeated compilation (and, for runtime performance-oriented objectives, execution) under many configurations. Second, optimization options are coarse-grained: disabling, enabling, or reordering a pass affects all occurrences of that transformation across the entire program. For example, if loop-invariant code motion is disabled at the option level

to avoid harming a particular loop (as in Fig. 1(a)), other loops where LICM would be beneficial are also affected.

用于编译器优化的自动调优。基于逐遍优化基础架构，一系列工作 (Liang 等人, 2023 年; Ansel 等人, 2014 年; Chen 等人, 2021 年; Zhu 等人, 2024 年; Park 等人, 2022 年; Pan 等人, 2025 年) 探索了自动调优方法，这些方法通过搜索编译器选项、优化标志或遍配置来提高性能。虽然这些方法可以带来性能提升，但它们存在几个固有限制。首先，搜索过程耗时，因为它需要在许多配置下重复编译 (以及对于面向运行时性能的目标进行执行)。其次，优化选项是粗粒度的：禁用、启用或重新排序一个遍会影响整个程序中该转换的所有出现情况。例如，如果在选项级别禁用循环不变代码移动以避免损害特定循环 (如图 1(a) 所示)，其他受益于 LICM 的循环也会受到影响。

LLMs for Compiler and Code Optimization. Recent work has begun to explore using LLMs for compiler construction and program optimization. Some approaches focus on enhancing compiler backend development efficiency (Zhong et al., 2024; 2025b;a), while others target source-level program rewriting for vectorization (Taneja et al., 2025). More closely related to our work are efforts that apply LLMs directly to IR-level optimization (Cummins et al., 2025; Yang et al., 2025), formulating optimization as an end-to-end generation task. While these methods bypass the traditional pass-by-pass pipeline, they typically rely on black-box generation without explicit representations of optimization intent. In contrast, our work models optimization intent as an explicit intermediate step and integrates LLM-based reasoning with compiler knowledge to enable globally coordinated IR optimization.

用于编译器和代码优化的大语言模型。最近的工作开始探索使用大语言模型进行编译器构建和程序优化。一些方法专注于提高编译器后端开发效率 (Zhong 等人, 2024 年; 2025b; a)，而其他方法则针对源级程序重写以进行向量化 (Taneja 等人, 2025 年)。与我们的工作更密切相关的是将大语言模型直接应用于中间表示 (IR) 级优化的努力 (Cummins 等人, 2025 年; Yang 等人, 2025 年)，将优化表述为端到端生成任务。虽然这些方法绕过了传统的逐遍管道，但它们通常依赖黑盒生成，没有优化意图的显式表示。相比之下，我们的工作将优化意图建模为显式中间步骤，并将基于大语言模型的推理与编译器知识集成，以实现全局协调的 IR 优化。

3. Proposed Method: INTOPT

3. 提出的方法:INTOPT

To explicitly model and reason about optimization intent during compiler optimization, we propose INTOPT, an intent-driven IR optimization framework. As illustrated in Fig. 2, INTOPT contains three stages: intent formulation (Sec. 3.1), intent refinement (Sec. 3.2), and intent realization (Sec. 3.3). This structure decouples high-level optimization intent from low-level analysis and transformations, allowing INTOPT to make globally coordinated IR optimization.

为了在编译器优化过程中显式地对优化意图进行建模和推理，我们提出了 INTOPT，一种意图驱动的 IR 优化框架。如图 2 所示，INTOPT 包含三个阶段：意图制定 (第 3.1 节)、意图细化 (第 3.2 节) 和意图实现 (第 3.3 节)。这种结构将高级优化意图与低级分析和转换解耦，使 INTOPT 能够进行全局协调的 IR 优化。

3.1. Intent Formulation

3.1. 意图制定

The intent formulation stage generates an initial optimization strategy S , which serves as a structured instantiation of the optimization intent, for a given input IR program P :

意图制定阶段为给定的输入 IR 程序 P 生成初始优化策略 S ，它作为优化意图的结构化实例:

$$S = \mathcal{F}_\theta(P)$$

where $\mathcal{F}_\theta$ denotes an optimization-specialized LLM parameterized by θ , fine-tuned on compiler optimization behaviors. The strategy S is represented as an ordered sequence of transformation actions $\langle a_1, a_2, \dots, a_n \rangle$, where each action a_i specifies in natural language which transformation to apply to which program region (e.g., "Vectorize line L_3 with <2 x double>").

其中 $\mathcal{F}_\theta$ 表示由 θ 参数化的优化专用大语言模型，在编译器优化行为上进行了微调。策略 S 表示为转换动作 $\langle a_1, a_2, \dots, a_n \rangle$ 的有序序列，其中每个动作 a_i 用自然语言指定对哪个程序区域应用哪种转换 (例如，“用 <2 x double> 向量化第 L_3 行”)。

Representing actions in natural language avoids prematurely constraining the optimization action space, allowing the model to reason over a richer and more flexible space of optimization strategies. Despite this flexibility, the problem remains tractable: while the space of semantically equivalent IR programs is vast, the set of transformation actions exposed by a modern compiler is finite and curated by compiler developers. This bounded action set makes intent formulation amenable to data-driven learning from a modestly sized dataset. Consequently, to fine-tune the intent formulation model $\mathcal{F}_\theta$, we construct a dataset by distilling optimization intents from compiler behaviors.

用自然语言表示动作避免了过早限制优化动作空间，使模型能够在更丰富、更灵活的优化策略空间中进行推理。尽管有这种灵活性，问题仍然是可处理的: 虽然语义等效的 IR 程序空间很大，但现代编译器公开的转换动作集是有限的，并且由编译器开发人员精心策划。这个有界动作集使得意图制定适合从适度大小的数据集中进行数据驱动的学习。因此，为了微调意图制定模型 $\mathcal{F}_\theta$ ，我们通过从编译器行为中提取优化意图来构建一个数据集。

Dataset Construction. As shown in Fig. 2(a), we first collect a diverse set of IR programs from IR-OptSet (Yang et al., 2025), an IR-oriented dataset designed to trigger representative optimization transformations encountered in real compiler optimization scenarios. We compile the collected programs using LLVM 19.1.0 (LLVM, 2026a) with the -O3 flag to obtain paired unoptimized and optimized IR programs $(P^{\text{unopt}}, P^{\text{opt}})$, where -O3 enables a broad range of optimization passes. We then distill optimization intent from the observed end-to-end optimization behavior P^{unopt} to P^{opt} by using GPT-5 (OpenAI, 2025) to infer the underlying optimization strategy S for each IR pair. The resulting triples - unoptimized IR P^{unopt} , optimized IR P^{opt} , and inferred strategy S - constitute the training dataset $\mathcal{D}_{\text{train}}$.

数据集构建。如图 2(a) 所示，我们首先从 IR - OptSet(Yang 等人, 2025 年) 收集了一组多样化的中间表示 (IR) 程序，IR - OptSet 是一个面向 IR 的数据集，旨在触发实际编译器优化场景中遇到的代表性优化转换。我们使用带有 -O3 标志的 LLVM 19.1.0(LLVM, 2026a) 编译收集到的程序，以获得成对的未优化和优化后的 IR 程序 ($P^{\text{unopt}}, P^{\text{opt}}$)，其中 -O3 启用了广泛的优化 passes。然后，我们通过使用 GPT - 5(OpenAI, 2025 年) 从观察到的端到端优化行为 P^{unopt} 中提取优化意图到 P^{opt} ，以推断每个 IR 对的潜在优化策略 S 。得到的三元组——未优化的 IR P^{unopt} 、优化后的 IR P^{opt} 和推断出的策略 S ——构成了训练数据集 $\mathcal{D}_{\text{train}}$ 。

Model Training. We fine-tune LLM Compiler FTD 13B (Cummins et al., 2025), an LLM for IR optimization, on $\mathcal{D}_{\text{train}}$ to obtain the optimization-specialized LLM $\mathcal{F}_{\theta}$. The training objective is to minimize the negative log-likelihood of generating the ground-truth optimization strategy S conditioned on the unoptimized IR program P^{unopt} :

模型训练。我们在 $\mathcal{D}_{\text{train}}$ 上对 LLM Compiler FTD 13B(Cummins 等人, 2025 年) 进行微调，LLM Compiler FTD 13B 是一个用于 IR 优化的语言模型，以获得优化专用的语言模型 $\mathcal{F}_{\theta}$ 。训练目标是最小化在未优化的 IR 程序 P^{unopt} 条件下生成真实优化策略 S 的负对数似然：

$$\mathcal{L}(\theta) = -\mathbb{E}_{(P^{\text{unopt}}, S) \sim \mathcal{D}_{\text{train}}} [\log p_{\theta}(S | P^{\text{unopt}})]$$

The choice of a 13B-parameter model is deliberate: intent formulation is primarily pattern recognition over program structures and compiler optimization behaviors, which does not require the extensive reasoning capabilities needed for subsequent refinement and realization stages. This design balances effectiveness with computational efficiency.

选择 13B 参数模型是经过深思熟虑的：意图制定主要是对程序结构和编译器优化行为的模式识别，这不需要后续细化和实现阶段所需的广泛推理能力。这种设计在有效性和计算效率之间取得了平衡。

3.2. Intent Refinement

3.2 意图细化

The intent formulation stage produces an initial optimization strategy that specifies which transformations to apply and in what order. Although this strategy captures high-level optimization intent learned from compiler behaviors, it is inferred in a data-driven manner and may miss key constraints from program semantics or target-specific requirements. The intent refinement stage therefore grounds the initial strategy in concrete compiler analysis and revises it when needed, ensuring that each proposed transformation is applicable and well-informed.

意图制定阶段产生一个初始优化策略，该策略指定要应用哪些转换以及按什么顺序应用。尽管此策略捕获了从编译器行为中学到的高级优化意图，但它是通过数据驱动的方式推断出来的，可能会错过程序语义或特定目标要求中的关键约束。因此，意图细化阶段将初始策略基于具体的编译器分析，并在必要时进行修订，确保每个提议的转换都是适用的且信息充分。

A key challenge of intent refinement is identifying which compiler analysis is necessary to refine the given strategy S . Since different transformations rely on different types of analysis, retrieving all available analysis is both inefficient and unnecessary.

意图细化的一个关键挑战是确定细化给定策略 S 需要哪些编译器分析。由于不同的转换依赖于不同类型的分析，检索所有可用分析既低效又不必要。

To address this challenge, we design an analysis-aware retrieval mechanism that explicitly captures the relationship between transform passes and their required analysis. By constructing a structured compiler knowledge base and using it to retrieve only the analysis relevant to each transformation action in S , the refinement process concentrates on the most informative analysis signals while avoiding unnecessary overhead.

为了应对这一挑战，我们设计了一种分析感知检索机制，该机制明确捕获转换 passes 与其所需分析之间的关系。通过构建一个结构化的编译器知识库并使用它仅检索与 S 中每个转换动作相关的分析，细化过程专注于最有信息价值的分析信号，同时避免不必要的开销。

Compiler Knowledge Base Construction. We construct a structured knowledge base $\mathcal{K}$ that catalogs compiler optimization passes (i.e., transform passes) and their analysis dependencies. Each entry $k \in \mathcal{K}$ is represented as:

编译器知识库构建。我们构建一个结构化知识库 $\mathcal{K}$ ，该知识库对编译器优化 passes(即转换 passes) 及其分析依赖项进行编目。每个条目 $k \in \mathcal{K}$ 表示为:

$$k = (ID, Desc, Deps),$$

where ID uniquely identifies a transform pass (corresponding to its pass class name, e.g., `LoopVectorizePass`), $Desc$ provides a natural-language description of the pass's functionality and optimization objectives, and $Deps$ denotes the set of compiler analysis required by the pass.

其中 ID 唯一标识一个转换 pass(对应于其 pass 类名，例如 `LoopVectorizePass`)， $Desc$ 提供该 pass 功能和优化目标的自然语言描述， $Deps$ 表示该 pass 所需的编译器分析集。

We build $\mathcal{K}$ through an automatic extraction process from the LLVM compiler infrastructure. First, we parse `llvm/lib/Passes/PassRegistry.def`, which enumerates all transform passes, to obtain the set of pass identifiers ID . For each pass, we then extract its functional description from the official LLVM pass documentation¹ to populate the corresponding $Desc$ field.

我们通过与 LLVM 编译器基础设施的自动提取过程构建 $\mathcal{K}$ 。首先，我们解析 `llvm/lib/Passes/PassRegistry.def`，它枚举了所有转换 passes，以获得 pass 标识符集 ID 。然后，对于每个 pass，我们从官方 LLVM pass 文档¹中提取其功能描述，以填充相应的 $Desc$ 字段。

To extract the analysis dependencies $Deps$, we exploit a key design principle of LLVM: each transform pass must explicitly declare analysis passes it relies on by invoking `AnalysisManager::getResult` within its run method. Accordingly, LLVM provides a collection of analysis passes, each responsible for extracting specific

optimization-related information from the IR programs to guide transformation decisions. This explicit dependency mechanism allows us to systematically trace which analysis passes are required by each transform pass, yielding a reliable and complete specification of its analysis requirements. For example, LoopVectorizePass requests loop-structure and target-specific analysis as follows:

为了提取分析依赖项 Deps, 我们利用 LLVM 的一个关键设计原则: 每个转换 pass 必须通过在其 run 方法中调用 AnalysisManager::getResult 来明确声明它依赖的分析 passes。因此, LLVM 提供了一组分析 passes, 每个分析 pass 负责从 IR 程序中提取特定的与优化相关的信息, 以指导转换决策。这种明确的依赖机制使我们能够系统地追踪每个转换 pass 需要哪些分析 passes, 从而可靠且完整地指定其分析要求。例如, LoopVectorizePass 如下请求循环结构和特定目标分析:

```
Preservedanalysis LoopVectorizePass::run(...,
```

```
Preservedanalysis LoopVectorizePass::run(...,
```

```
AnalysisManager &AM) {
```

```
AnalysisManager &AM) {
```

```
...
```

```
https://llvm.org/docs/Passes.html
```

```
auto &LI = AM.getResult<LoopAnalysis>(F);
```

```
auto &LI = AM.getResult<LoopAnalysis>(F);
```

```
auto &TLI = AM.getResult<TargetLibraryAnalysis>(F);
```

```
auto &TLI = AM.getResult<TargetLibraryAnalysis>(F);
```

```
}
```

Therefore, we obtain the Deps field by locating each pass's run implementation under llvm/lib/ Transforms and statically parsing all invocations of AnalysisManager::getResult. As shown in Fig. 2(b), the dependency set for LoopVectorizePass is {LoopAnalysis, TargetLibraryAnalysis}.

因此, 我们通过在 llvm/lib/Transforms 下定位每个通行证的运行实现, 并静态解析 AnalysisManager::getResult 的所有调用, 来获取 Deps 字段。如图 2(b) 所示, LoopVectorizePass 的依赖集为 {LoopAnalysis, TargetLibraryAnalysis}。

Retrieve Analysis Results. With the constructed compiler knowledge base $\mathcal{K}$, we retrieve the concrete compiler transform passes and their dependent analysis passes corresponding to each transformation action a_i in the initial optimization strategy S . Specifically, for each action a_i , we treat the natural-language description of a_i as a query and match it against the document corpus $\{\text{Desc}(k)\}_{k \in \mathcal{K}}$. Both the query and documents are

represented using TF-IDF (Zhang et al., 2011) with 1-3 gram tokenization, and the top-m matching passes are selected based on cosine similarity.

检索分析结果。利用构建的编译器知识库 $\mathcal{K}$ ，我们检索与初始优化策略 S 中每个转换动作 a_i 相对应的具体编译器转换通行证及其依赖的分析通行证。具体来说，对于每个动作 a_i ，我们将 a_i 的自然语言描述作为查询，并将其与文档语料库 $\{\text{Desc}(k)\}_{k \in \mathcal{K}}$ 进行匹配。查询和文档都使用带有 1-3 克分词的 TF-IDF (Zhang 等人, 2011) 表示，并且基于余弦相似度选择前 m 个匹配的通行证。

$$\text{sim}(a_i, k) = \cos(\text{TF-IDF}(a_i), \text{TF-IDF}(\text{Desc}(k)))$$

$$\{k_i^{(j)}\}_{j=1}^m = \arg \text{top-} m_{k \in \mathcal{K}} \text{sim}(a_i, k)$$

Such design balances efficiency and coverage: top-m retrieval captures cases where multiple passes address similar goals (e.g., EarlyCSEPass, DCEPass, and GVNPASS all perform redundancy elimination) while avoiding the computational overhead of indiscriminate analysis execution.

这样的设计在效率和覆盖范围之间取得了平衡: 前 m 检索捕获了多个通行证解决相似目标的情况 (例如, EarlyCSEPass、DCEPass 和 GVNPASS 都执行冗余消除), 同时避免了无差别分析执行的计算开销。

Once the passes $\{k_i^{(j)}\}_{j=1}^m$ are identified, we retrieve their required analysis passes $\text{Deps}(k_i^{(j)})$ from the knowledge base and invoke the LLVM compiler to generate the corresponding analysis results on the current IR program P :

一旦确定了通行证 $\{k_i^{(j)}\}_{j=1}^m$ ，我们就从知识库中检索它们所需的分析通行证 $\text{Deps}(k_i^{(j)})$ ，并调用 LLVM 编译器在当前 IR 程序 P 上生成相应的分析结果:

$$R_S = \bigcup_{a_i \in S} \bigcup_{j=1}^m \text{LLVM_Generate}(\text{Deps}(k_i^{(j)}), P)$$

The analysis results are obtained through LLVM’s analysis inspection interfaces (e.g., `opt -p=print<analysis>`), which generate analysis information in textual form. This information corresponds to the same analysis results that transform passes query during the compiler optimization process to ensure correctness and performance. As a result, they provide reliable information for downstream intent refinement and intent realization.

分析结果是通过 LLVM 的分析检查接口 (例如, `opt -p=print<analysis>`) 获得的, 这些接口以文本形式生成分析信息。此信息与转换通行证在编译器优化过程中查询的相同分析结果相对应, 以确保正确性和性能。因此, 它们为下游意图细化和意图实现提供了可靠的信息。

Refinement. Given the initial optimization strategy S and the retrieved analysis results R_S , the refinement stage produces a refined strategy $\hat{S}$:

细化。给定初始优化策略 S 和检索到的分析结果 R_S ，细化阶段产生一个细化策略 $\hat{S}$:

$$\hat{S} = \mathcal{G}(P, S, R_S)$$

where $\mathcal{G}$ denotes the LLM (GPT-5) that performs intent refinement by reasoning over the program, initial strategy, and compiler analysis information. The prompt template used during refinement is provided in Appendix B.

其中 $\mathcal{G}$ 表示通过对程序、初始策略和编译器分析信息进行推理来执行意图细化的 LLM(GPT-5)。细化期间使用的提示模板在附录 B 中提供。

This refinement process validates and enhances the initial strategy in Section 3.1 by grounding it in compiler analysis information: it filters infeasible transformations based on program constraints, while leveraging both analysis information and the LLM’s pretraining knowledge to refine the transformations in ways that may exceed conventional compiler heuristics. The resulting strategy $\hat{S}$ is constraint-aware and analytically informed, providing a reliable bridge between intent formulation and concrete intent realization.

此细化过程通过将其基于编译器分析信息进行验证和增强，在第 3.1 节中的初始策略：它根据程序约束过滤不可行的转换，同时利用分析信息和 LLM 的预训练知识以可能超出传统编译器启发式的方式细化转换。得到的策略 $\hat{S}$ 是约束感知的且经过分析告知的，在意图制定和具体意图实现之间提供了可靠的桥梁。

3.3. Intent Realization

3.3. 意图实现

The intent realization stage finally applies the strategy $\hat{S}$ on the unoptimized IR program P to generate the optimized IR program $\hat{P}$:

意图实现阶段最终将策略 $\hat{S}$ 应用于未优化的 IR 程序 P 以生成优化的 IR 程序 $\hat{P}$ ：

$$\hat{P} = \mathcal{G}(P, \hat{S}, R_S)$$

where $\mathcal{G}$ denotes the same LM (GPT-5) used in refinement. This stage synthesizes the optimized program by applying the transformation actions specified in $\hat{S}$ while respecting the constraints encoded in the analysis results R_S . The prompt template is provided in Appendix B.

其中 $\mathcal{G}$ 表示在细化中使用的相同 LM(GPT-5)。此阶段通过应用 $\hat{S}$ 中指定的转换动作来合成优化程序，同时尊重分析结果 R_S 中编码的约束。提示模板在附录 B 中提供。

Through this three-stage pipeline - formulation, refinement, and realization - INTOPT transforms high-level optimization intent into concrete, well-informed IR transformations. Leveraging GPT-5 for intent refinement and realization allow INTOPT to explore optimization strategies beyond those typically produced by traditional fixed-pass compilers.

通过这个三阶段管道——制定、细化和实现——INTOPT 将高级优化意图转换为具体的、信息丰富的 IR 转换。利用 GPT-5 进行意图细化和实现使 INTOPT 能够探索超出传统固定通行证编译器通常产生的优化策略。

4. Experiments

4. 实验

This section explores the following research questions:

本节探讨以下研究问题:

- RQ.1 Can INTOPT outperform existing end-to-end LLM-based IR optimization approaches? (Sec. 4.2)
- RQ.1 INTOPT 能否优于现有的基于端到端 LLM 的 IR 优化方法? (第 4.2 节)
- RQ.2 How does INTOPT compare against the traditional compiler in terms of performance? Does the explicit optimization intent lead to performance gains beyond the pass-by-pass paradigm? (Sec. 4.3 & Sec. 4.4)
- RQ.2 INTOPT 在性能方面与传统编译器相比如何? 明确的优化意图是否会带来超出逐通行证范式的性能提升? (第 4.3 节和第 4.4 节)
- RQ.3 How does each component of INTOPT contribute to its overall results? (Sec. 4.5)
- 研究问题 3:INTOPT 的每个组件如何对其整体结果产生贡献? (第 4.5 节)

4.1. Experimental Setup

4.1. 实验设置

4.1.1. DATASET

4.1.1. 数据集

We construct all datasets from IR-OptSet (Yang et al., 2025), an IR-oriented dataset designed to trigger representative optimization transformations encountered in real compiler optimization scenarios, and each IR sample (unoptimized IR plus optimized IR) is capped at 5,000 tokens. We randomly sample 4,000 programs to form the training set $\mathbf{INT}_{\text{train}}$ and 500 programs to form the validation set $\mathbf{INT}_{\text{val}}$. For evaluation, we construct a separate test set $\mathbf{INT}_{\text{test}}$ of 200 IR samples through a multi-stage filtering process. First, we retain programs for which Alive2 (Lopes et al., 2021) can verify semantic equivalence between the unoptimized IR and LLVM -03. Second, we exclude programs unsuitable for differential testing (McKeeman, 1998), such as

those with custom external functions, externally defined global variables, or non-deterministic behavior. This ensures that when LLM-based optimizers produce transformations beyond Alive2’s verification capability, differential testing can be reliably used for correctness validation.

我们从 IR - OptSet(Yang 等人, 2025 年) 构建所有数据集, IR - OptSet 是一个面向 IR 的数据集, 旨在触发实际编译器优化场景中遇到的代表性优化转换, 并且每个 IR 样本(未优化的 IR 加上优化后的 IR) 限制为 5000 个令牌。我们随机抽取 4000 个程序形成训练集 $\mathbf{INT}_{\text{train}}$, 500 个程序形成验证集 $\mathbf{INT}_{\text{val}}$ 。为了进行评估, 我们通过多阶段过滤过程构建了一个由 200 个 IR 样本组成的单独测试集 $\mathbf{INT}_{\text{test}}$ 。首先, 我们保留 Alive2(Lopes 等人, 2021 年) 能够验证未优化 IR 与 LLVM - 03 之间语义等价性的程序。其次, 我们排除不适合差异测试(McKeeman, 1998 年) 的程序, 例如那些具有自定义外部函数、外部定义的全局变量或非确定性行为的程序。这确保了当基于大语言模型的优化器产生超出 Alive2 验证能力的转换时, 差异测试可以可靠地用于正确性验证。

Table 1. Comparison of correctness and performance of six LLM-based IR optimization methods. Correctness is evaluated using Alive2 combined with differential testing, while performance is measured as speedup over the input unoptimized IR.

表 1. 六种基于大语言模型的 IR 优化方法的正确性和性能比较。正确性使用 Alive2 结合差异测试进行评估, 而性能则以相对于输入未优化 IR 的加速比来衡量。

Method	Correctness		Performance			
	Alive2	Alive2+Diff. Test.	Avg. Speedup	Speedup > 1.1×	Speedup > 1.5×	Speedup > 2.0×
GPT-5	73.0% (146)	83.5% (167)	1.057×	26.0% (52)	10.5% (21)	6.5% (13)
Claude Haiku 4.5	54.0% (108)	61.0% (122)	0.721×	15.5% (31)	5.0% (10)	2.5% (5)
DeepSeek-V3.2	51.0% (102)	59.0% (118)	0.719×	14.5% (29)	6.5% (13)	3.5% (7)
LLM Compiler FTD 7B	57.0% (114)	60.5% (121)	0.702×	13.0% (26)	6.0% (12)	2.5% (5)
LLM Compiler FTD 13B	61.5% (123)	64.5% (129)	0.757×	12.0% (24)	4.0% (8)	2.5% (5)
INTOPT	71.0% (142)	90.5% (181)	2.660×	37.5% (75)	17.5% (35)	10.0% (20)

方法	正确性		性能			
	Alive2	Alive2+ 差异测试	平均加速比	加速比 > 1.1×	加速比 > 1.5×	加速比 > 2.0×
GPT-5	73.0% (146)	83.5% (167)	1.057 倍	26.0% (52)	10.5% (21)	6.5% (13)
Claude Haiku 4.5	54.0% (108)	61.0% (122)	0.721 倍	15.5% (31)	5.0% (10)	2.5% (5)
DeepSeek-V3.2	51.0% (102)	59.0% (118)	0.719 倍	14.5% (29)	6.5% (13)	3.5% (7)
LLM 编译器 FTD 7B	57.0% (114)	60.5% (121)	0.702 倍	13.0% (26)	6.0% (12)	2.5% (5)
LLM 编译器 FTD 13B	61.5% (123)	64.5% (129)	0.757 倍	12.0% (24)	4.0% (8)	2.5% (5)
INTOPT	71.0% (142)	90.5% (181)	2.660×	37.5% (75)	17.5% (35)	10.0% (20)

4.1.2. BASELINES

4.1.2. 基线

We include two categories of baselines for comparison:

我们纳入了两类基线用于比较:

End-to-end LLM-based IR optimizers. We consider both general-purpose and specialized LLM-based optimizers that directly map unoptimized IR to optimized IR. Specifically, we evaluate three representative

general-purpose LLMs with strong capabilities on code-related tasks: GPT-5 (OpenAI, 2025), Claude Haiku 4.5 (Anthropic, 2025), and DeepSeek-V3.2 (DeepSeek-AI et al., 2025). All models are prompted using the template shown in Appendix C. We also compare against LLM Compiler (Cummins et al., 2025), a state-of-the-art specialized end-to-end IR optimizer, evaluating both the 7B and 13B parameter variants (LLM Compiler FTD 7B and LLM Compiler FTD 13B). To ensure a fair comparison, both LLM Compiler variants are fine-tuned on $\text{INT}_{\text{train}}$ with INT_{val} following Appendix E.

基于端到端大语言模型的 IR 优化器。我们考虑了通用型和专用型基于大语言模型的优化器，它们直接将未优化的 IR 映射为优化后的 IR。具体而言，我们评估了在代码相关任务上具有强大能力的三个代表性通用大语言模型:GPT-5(OpenAI, 2025 年)、Claude Haiku 4.5(Anthropic, 2025 年)和 DeepSeek-V3.2(DeepSeek-AI 等, 2025 年)。所有模型均使用附录 C 中所示的模板进行提示。我们还与 LLM Compiler(Cummins 等, 2025 年)进行比较，它是一种先进的专用端到端 IR 优化器，评估了 7B 和 13B 参数变体 (LLM Compiler FTD 7B 和 LLM Compiler FTD 13B)。为确保公平比较，两个 LLM Compiler 变体均按照附录 E 中的方法在 $\text{INT}_{\text{train}}$ 上使用 INT_{val} 进行微调。

Compiler baseline. We include LLVM 19.1.0 (LLVM, 2026a) with the -O3 optimization level as a compiler-level reference. -O3 aggressively optimizes for execution speed (LLVM, 2026c) and represents a highly tuned IR optimization strategy.

编译器基线。我们纳入了优化级别为-O3 的 LLVM 19.1.0(LLVM, 2026a) 作为编译器级别的参考。-O3 针对执行速度进行积极优化 (LLVM, 2026c)，代表了一种高度调优的 IR 优化策略。

4.1.3. IMPLEMENTATION DETAILS

4.1.3. 实现细节

For the intent formulation stage of INTOPT, we obtain the optimization-specialized LLM $\mathcal{F}_\theta$ in INTOPT, using the $\text{INT}_{\text{train}}$ with INT_{val} , following the procedure described in Sec. 3.1. During the analysis results retrieval stage, we select the top-3 matching optimization passes and collect their corresponding LLVM analysis results, which are used to support the intent refinement stage.

对于 INTOPT 的意图制定阶段，我们按照 3.1 节中描述的过程，在 INTOPT 中使用 $\text{INT}_{\text{train}}$ 和 INT_{val} 获得优化专用的大语言模型 $\mathcal{F}_\theta$ 。在分析结果检索阶段，我们选择前 3 个匹配的优化 passes 并收集它们相应的 LLVM 分析结果，这些结果用于支持意图细化阶段。

4.1.4. EVALUATION METRICS

4.1.4. 评估指标

Correctness Verification. We employ a two-stage verification process to validate the Correctness of optimized IR programs. We first apply Alive2 to perform strict formal semantic equivalence checking between the optimized and unoptimized IR. Since Alive2 may time out on verifying programs with complex control flow or memory behaviors (Lopes et al., 2021), we additionally apply differential testing when formal verification is infeasible. Specifically, we use LibFuzzer (LLVM, 2026b) to generate diverse test inputs and verify that

optimized and unoptimized programs produce identical outputs across different execution paths. Details are provided in Appendix F.

正确性验证。我们采用两阶段验证过程来验证优化后的 IR 程序的正确性。我们首先应用 Alive2 对优化后的 IR 和未优化的 IR 之间进行严格的形式语义等价性检查。由于 Alive2 在验证具有复杂控制流或内存行为的程序时可能会超时 (Lopes 等, 2021), 当形式验证不可行时, 我们还应用差异测试。具体而言, 我们使用 LibFuzzer(LLVM, 2026b) 生成各种测试输入, 并验证优化后的程序和未优化的程序在不同执行路径上产生相同的输出。详细信息在附录 F 中提供。

Performance Evaluation. We measure performance using the same inputs generated by LibFuzzer during correctness verification, reflecting execution behaviors across diverse control flow paths. Each program is executed for 10,000 iterations to obtain stable timing measurements and mitigate measurement noise. We report Performance as speedup ratio: $\text{Speedup} = \text{RunTime}(\text{unopt})/\text{RunTime}(\text{opt})$. When an LLM-based optimizer generates incorrect IR, we assign a speedup of 0. More details are in Appendix G.

性能评估。我们使用正确性验证期间由 LibFuzzer 生成的相同输入来测量性能, 反映不同控制流路径上的执行行为。每个程序执行 10,000 次迭代以获得稳定的定时测量并减轻测量噪声。我们将性能报告为加速比: $\text{加速比} = \text{RunTime}(\text{未优化})/\text{RunTime}(\text{优化})$ 。当基于大语言模型的优化器生成不正确的 IR 时, 我们将加速比设为 0。更多详细信息在附录 G 中。

4.2. Improvement over LLM-based IR Optimizers

4.2. 相对于基于大语言模型的 IR 优化器的改进

We first compare INTOPT with end-to-end LLM-based IR optimizers on the 200-sample benchmark INT_{test} . As shown in Table 1, INTOPT achieves the strongest overall results across all baselines in both correctness and performance.

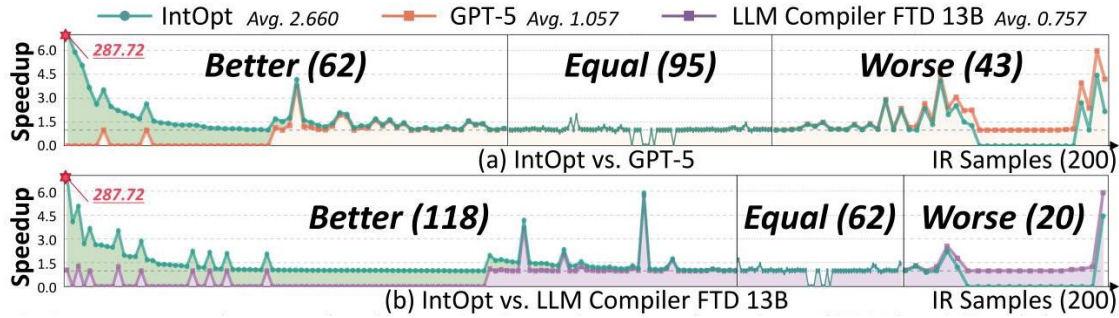
我们首先在 200 样本基准 INT_{test} 上比较 INTOPT 和基于端到端大语言模型的 IR 优化器。如表 1 所示, INTOPT 在正确性和性能方面在所有基线中取得了最强的总体结果。

Correctness. Under Alive2-only verification, GPT-5 achieves the highest correctness rate among baselines (73.0%), while INTOPT attains a comparable rate of 71.0%. However, this metric is inherently limited by Alive2’s verification capability: even complex optimizations produced by LLVM -03 cannot always be verified (Lopes et al., 2021). When differential testing is incorporated, INTOPT achieves a verified correctness rate of 90.5%, surpassing all other baselines by 7.0%-31.5%. These results highlight INTOPT’s advantage in reducing semantic errors over end-to-end LLM-based IR optimization methods.

正确性。仅在 Alive2 验证下, GPT-5 在基线中实现了最高的正确率 (73.0%), 而 INTOPT 达到了可比的 71.0%。然而, 该指标本质上受 Alive2 验证能力的限制: 即使是 LLVM -03 产生的复杂优化也并非总能被验证 (Lopes 等, 2021)。当纳入差异测试时, INTOPT 实现了 90.5% 的验证正确率, 比所有其他基线高出 7.0%-31.5%。这些结果突出了 INTOPT 在减少语义错误方面相对于基于端到端大语言模型的 IR 优化方法的优势。

Performance. INTOPT achieves an average speedup of $2.660\times$ over the unoptimized IR, significantly outperforming all end-to-end LLM-based baselines. The strongest base-

性能。INTOPT 相较于未优化的中间表示 (IR) 实现了平均 $2.660\times$ 的加速，显著优于所有基于端到端大语言模型 (LLM) 的基线方法。最强的基础



differences within $\pm 2\%$ are considered equal to account for runtime variability. line, GPT-5, attains an average speedup of only $1.057\times$, resulting in a $1.603\times$ performance gap. Beyond average performance, INTOPT also consistently dominates across all speedup thresholds, indicating that its performance gains are broadly distributed across the benchmark.

考虑到运行时的可变性， $\pm 2\%$ 内的差异被视为相等。GPT-5 仅实现了平均 $1.057\times$ 的加速，导致了 $1.603\times$ 的性能差距。除了平均性能之外，INTOPT 在所有加速阈值上也始终占据优势，这表明其性能提升在整个基准测试中广泛分布。

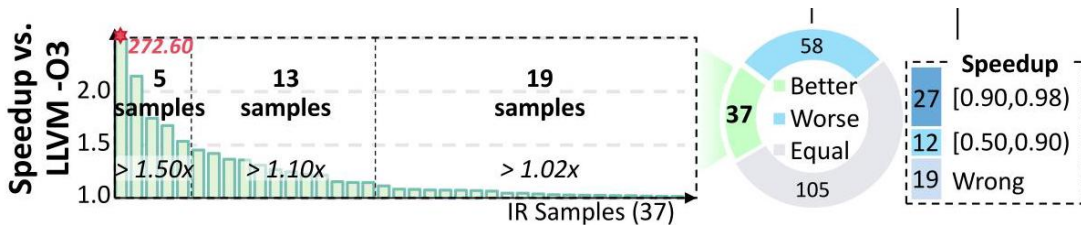


Figure 4. Performance comparison between INTOPT and LLVM -03. Speedup is defined as $\text{RunTime}(\text{LLVM -03})/\text{RunTime}(\text{opt})$, with $[0.98, 1.02]$ treated as equal. INTOPT matches LLVM -03 on 105 programs and outperforms it on 37 (max $272.60\times$); among 58 underperforming cases, most are slight slowdowns (27 in $[0.90, 0.98]$), and 19 are incorrect.

图 4. INTOPT 与 LLVM -03 的性能比较。加速比定义为 $\text{RunTime}(\text{LLVM -03})/\text{RunTime}(\text{opt})$ ，其中 $[0.98, 1.02]$ 视为相等。INTOPT 在 105 个程序上与 LLVM -03 匹配，在 37 个程序上优于它（最大 272.60 倍）；在 58 个性能较差的案例中，大多数是轻微减速（27 个在 $[0.90, 0.98]$ ），19 个是错误的。

To better understand these improvements, we further compare INTOPT against the two strongest baselines, GPT-5 and LLM Compiler FTD 13B, on a per-program basis, as shown in Fig. 3. Against GPT-5, INTOPT achieves higher speedups on 62 programs, while the advantage is more pronounced against LLM Compiler FTD 13B, where INTOPT outperforms it on 118 programs. Notably, when compared to the unoptimized IR,

INTOPT attains a maximum speedup of up to $287.72\times$ on a single program. These results suggest that INTOPT is more effective at discovering and realizing aggressive yet profitable optimization opportunities.

为了更好地理解这些改进，我们在每个程序的基础上进一步将 INTOPT 与两个最强的基线 GPT-5 和 LLM Compiler FTD 13B 进行比较，如图 3 所示。与 GPT-5 相比，INTOPT 在 62 个程序上实现了更高的加速比，而在与 LLM Compiler FTD 13B 的比较中优势更明显，INTOPT 在 118 个程序上优于它。值得注意的是，与未优化的 IR 相比，INTOPT 在单个程序上实现了高达 $287.72\times$ 的最大加速比。这些结果表明，INTOPT 在发现和实现激进但有利可图的优化机会方面更有效。

Answer to RQ.1: By decoupling optimization intent formulation from concrete transformation generation, INTOPT reduces semantic errors and is more effective at discovering profitable optimization opportunities than existing end-to-end LLM-based IR optimization approaches.

对 RQ.1 的回答: 通过将优化意图制定与具体转换生成解耦，INTOPT 减少了语义错误，并且在发现有利可图的优化机会方面比现有的基于端到端 LLM 的 IR 优化方法更有效。

4.3. Improvement over Traditional Compiler

4.3. 相对于传统编译器的改进

We further compare INTOPT's performance against LLVM -03, a highly optimized and widely adopted compiler optimization level. Fig. 4 shows the comparison results.

我们进一步将 INTOPT 的性能与 LLVM -03 进行比较，LLVM -03 是一个高度优化且广泛采用的编译器优化级别。图 4 显示了比较结果。

INTOPT achieves performance comparable to LLVM -03 on 105 programs and outperforms it on 37 programs. Among these improved cases, the per-program speedup distribution reveals a spectrum of performance gains: 5 programs exhibit substantial improvements with speedups greater than $1.5\times$, while 13 programs achieve clear improvements exceeding $1.1\times$. Notably, the program that achieves the highest speedup in Fig. 3 also attains the highest performance improvement over LLVM -03, with a speedup of up to $272.60\times$. We analyze this case in detail in Sec. 4.4, illustrating how INTOPT can uncover optimization opportunities that go beyond the capabilities of the traditional compiler.

INTOPT 在 105 个程序上实现了与 LLVM -03 相当的性能，在 37 个程序上优于它。在这些改进的案例中，每个程序的加速比分布显示了一系列性能提升: 5 个程序表现出显著改进，加速比大于 $1.5\times$ ，而 13 个程序实现了明显改进，超过 $1.1\times$ 。值得注意的是，图 3 中实现最高加速比的程序在相对于 LLVM -03 的性能提升方面也最高，加速比高达 272.60 倍。我们在 4.4 节中详细分析了这个案例，说明了 INTOPT 如何发现超越传统编译器能力的优化机会。

4.4. Case Analysis

4.4. 案例分析

To provide deeper insights into the intent-driven optimization mechanism, we present two representative cases that illustrate how INTOPT explores optimization opportunities beyond the traditional pass-by-pass compiler paradigm. Due to space constraints, we present pseudo-code representations of the IR; the full code listings are provided in Appendix H. As shown in Fig. 5, traditional compilers primarily optimize programs by matching pre-defined patterns in passes designed by compiler developers and applying transformations (e.g., eliminating stack-based load/store operations).

为了更深入地了解意图驱动优化机制，我们展示两个代表性案例，说明 INTOPT 如何探索超越传统逐遍编译器范式的优化机会。由于篇幅限制，我们展示 IR 的伪代码表示；完整的代码清单在附录 H 中提供。如图 5 所示，传统编译器主要通过匹配编译器开发者设计的数据通路中的预定义模式并应用转换（例如，消除基于栈的加载/存储操作）来优化程序。

In contrast, INTOPT performs intent-guided optimization that enables coordinated, semantic-aware transformations. In *chocolateFeast*, INTOPT identifies an algorithmic shortcut during intent refinement that reduces the time complexity of an iterative computation to $O(1)$ and guides the subsequent optimization, achieving a 287.72 $\times$ speedup. In *reverse_bits*, INTOPT achieves a 1.97 $\times$ speedup by coordinating multiple transformations. After recognizing the high-level semantics of the loop as bit reversal, INTOPT replaces the original computation with the

相比之下，INTOPT 执行意图引导的优化，实现协调的、语义感知的转换。在 *chocolateFeast* 中，INTOPT 在意图细化过程中识别出一种算法捷径，将迭代计算的时间复杂度降低到 $O(1)$ ，并指导后续优化，实现了 287.72 $\times$ 倍的加速比。在 *reverse_bits* 中，INTOPT 通过协调多个转换实现了 1.97 $\times$ 倍的加速比。在识别出循环的高级语义为位反转后，INTOPT 用

Program Description: Computes the total chocolates obtainable when spending N dollars, where each chocolate costs C dollars and produces one wrapper, and M wrappers can be redeemed for one additional chocolate. Refined Strategy		
All values are first stored on the stack ('st') and then loaded for each operation.	Eliminate all stack-based load/store operations; Remove redundant ' $[N/C]$ ' computation.	When $M > 1$ and the init > 0 , the number of exchanges is ' $\lceil (init - 1) / (M - 1) \rceil$ '. Return 'init + $\lceil (init - 1) / (M - 1) \rceil$ ' without entering the loop. Time Complexity: $O(1)$
<pre>chocolateFeast(N, C, M): Input IR entry: #stack allocation st[3] = N, st[4] = C, st[5] = M st[6] = st[3] / st[4] # initial chocolates st[7] = st[3] / st[4] # initial wrappers loop: #not enough wrappers to redeem if (st[7] < st[5]) goto end st[7] = st[7] - st[5] # spend M wrappers st[6] = st[6] + 1 #gain one chocolate #gain one wrapper from the new chocolate st[7] = st[7] + 1 goto loop end: return st[6] #total chocolates</pre>	<pre>chocolateFeast(N, C, M): LLVM-03 entry: Speedup: 1.06x #initial chocolates (and wrappers) init = [N/C] #not enough wrappers to redeem if (init < M) return init loop: total = phi[init(entry), total'(loop)] wrap = phi[init(entry), wrap'(loop)] total' = total + 1 wrap' = wrap - M + 1 #not enough wrappers to redeem if (wrap' >= M) goto loop return total'</pre>	<pre>chocolateFeast(N, C, M): IntOpt entry: Speedup: 287.72x init = N / C if (M > 1 && init > 0) extra = $\lceil (init - 1) / (M - 1) \rceil$ return init + Extra if (init < M) return init loop: total = phi[init(entry), total'(loop)] wrap = phi[init(entry), wrap'(loop)] total' = total + 1 wrap' = wrap - M + 1 if (wrap' >= M) goto loop return total'</pre>
Program Description: Reverses the bit of a 32-bit integer N by iteratively extracting the lowest bit and building the result from left to right.		
All values are first stored on the stack ('st') and then loaded for each operation.	Eliminate stack-based load/store operations; Add an early-exit condition 'input < 2' to avoid redundant iterations.	Replace the loop with llvm intrinsic that backend can map to hardware: llvm.bitreverse.i32 to reverse all 32 bits, and llvm.cttz.i32 to compute the number of leading zeros.
<pre>reverse bits(N): Input IR entry: st[1] = N, st[2] = 0 # stack allocation loop: if (st[1] == 0) goto exit st[2] = st[2] * 2 # shift result left st[3] = st[1] & 1 # extract lowest bit st[2] = st[2] st[3] # append bit to result st[1] = st[1] >> 1 # shift input right goto loop exit: return st[2] #reversed bits</pre>	<pre>reverse bits(N): LLVM-03 entry: Speedup: 1.13x if (N == 0) return 0 loop: input = phi[N(entry), input'(loop)] result = phi[0(entry), result'(loop)] result = (result * 2) (input & 1) input' = input >> 1 #stop when input < 2 (input' must be 0) if (input < 2) return result' goto loop</pre>	<pre>reverse bits(N): IntOpt entry: Speedup: 1.97x if (N == 0) return 0 fast: Effective Utilization of Hardware Instructions #hardware bit-reverse intrinsic brev = call @llvm.bitreverse.i32(N) #find the leading zero clz = call @llvm.cttz.i32(N, false) #remove leading zeros from reversed res = brev >> clz return res</pre>

--	--	--

Figure 5. Case study illustrating how INTOPT explores optimization opportunities beyond the traditional pass-by-pass compiler paradigm.

图 5. 案例研究，说明 INTOPT 如何探索超越传统逐遍编译器范式的优化机会。

Table 2. Ablation study of INTOPT ’s design. Impact of intent refinement and compiler analysis on correctness and performance, where performance is measured as speedup over the input unoptimized IR.

表 2. INTOPT 设计的消融研究。意图细化和编译器分析对正确性和性能的影响，其中性能以相对于输入未优化 IR 的加速比来衡量。

Method	Correctness		Performance			
	Alive2	Alive2+Diff. Test.	Avg. Speedup	Speedup > 1.1×	Speedup > 1.5×	Speedup > 2.0×
INTOPT	71.0% (142)	90.5% (181)	2.660×	37.5% (75)	17.5% (35)	10.0% (20)
w/o Intent Refinement	71.5% (143)	87.0% (174)	1.122×	30.0% (60)	15.0% (30)	7.5% (15)
w/o Analysis	65.0% (130)	77.0% (154)	2.415×	29.0% (58)	13.0% (26)	6.5% (13)

方法	正确性		性能			
	Alive2	Alive2+ 差异测试	平均加速比	加速比 > 1.1×	加速比 > 1.5×	加速比 > 2.0×
INTOPT	71.0% (142)	90.5% (181)	2.660 倍	37.5% (75)	17.5% (35)	10.0% (20)
无意图细化	71.5% (143)	87.0% (174)	1.122 倍	30.0% (60)	15.0% (30)	7.5% (15)
无分析	65.0% (130)	77.0% (154)	2.415 倍	29.0% (58)	13.0% (26)	6.5% (13)

llvm.bitreverse intrinsic, and further applies an auxiliary transformation using llvm.ctlz with shift-based operations to model the original loop’s exit condition. These optimizations are difficult for existing compilers to realize, as they require semantic reasoning beyond what local pattern matching in a fixed pass-by-pass pipeline can capture.

llvm.bitreverse 内在函数，并进一步使用 llvm.ctlz 和基于移位的操作应用辅助转换，以模拟原始循环的退出条件。这些优化对于现有编译器来说很难实现，因为它们需要超出固定逐遍流水线中局部模式匹配所能捕获的语义推理。

Answer to RQ.2: Explicit optimization intent enables IN-TOPT to realize optimizations that go beyond the capabilities of traditional pass-by-pass compilers.

对 RQ.2 的回答: 显式优化意图使 IN-TOPT 能够实现超越传统逐遍编译器能力的优化。

4.5. Effectiveness of INTOPT ’s Design

4.5. INTOPT 设计的有效性

The intent formulation stage is a necessary first stage in this pipeline, while intent realization serves as the final stage that generates optimized IR; neither can be bypassed. Accordingly, our ablation study focuses on the contribution of the intent refinement stage and the impact of compiler analysis on INTOPT. As shown in Table 2, removing intent refinement slightly improves Alive2-only correctness but leads to a noticeable degradation in both verified correctness and performance, reducing average speedup from 2.660×

. Removing compiler analysis from both refinement and realization (w/o Analysis) further reduces verified correctness to 77.0%, as refinement and realization decisions are made without sufficient semantic constraints. Despite this, performance remains relatively high ($2.415\times$ on average). This behavior can be attributed to the fact that, starting from compiler-derived optimization intents produced during intent formulation, the LLM can occasionally identify highly aggressive transformations, including the case shown in Fig. 3 that achieves a speedup of $287.72\times$.

意图制定阶段是此流水线中必要的第一阶段，而意图实现是生成优化 IR 的最后阶段；两者都不能跳过。因此，我们的消融研究重点关注意图细化阶段的贡献以及编译器分析对 INTOPT 的影响。如表 2 所示，去除意图细化会略微提高仅 Alive2 的正确性，但会导致验证正确性和性能显著下降，将平均加速比从 $2.660\times$ 降至 $1.122\times$ 。从细化和实现中都去除编译器分析 (无分析) 会进一步将验证正确性降低到 77.0%，因为细化和实现决策是在没有足够语义约束的情况下做出的。尽管如此，性能仍然相对较高 (平均为 $2.415\times$)。这种行为可以归因于这样一个事实，即从意图制定期间生成的编译器派生优化意图开始，大语言模型偶尔可以识别高度激进的转换，包括图 3 所示的实现加速比为 $287.72\times$ 的情况。

Answer to RQ.3: Both intent refinement and compiler analysis are both critical to INTOPT, enabling effective optimization discovery and reliable realization.

对 RQ.3 的回答: 意图细化和编译器分析对 INTOPT 都至关重要，能够实现有效的优化发现和可靠的实现。

5. Conclusion

5. 结论

This paper introduces INTOPT, the first intent-driven IR optimization framework. By decoupling high-level optimization intent from low-level analysis and transformation, INTOPT addresses a critical yet underexplored dimension of IR optimization. Our experimental results show that INTOPT delivers substantial improvements over both end-to-end LLM-based optimizers and traditional compiler optimizations, demonstrating that making optimization intent explicit is crucial for achieving globally coordinated, semantically sound, and high-performance IR optimization.

本文介绍了 INTOPT，第一个意图驱动的 IR 优化框架。通过将高级优化意图与低级分析和转换解耦，INTOPT 解决了 IR 优化中一个关键但未充分探索的维度。我们的实验结果表明，INTOPT 相对于基于大语言模型的端到端优化器和传统编译器优化都有显著改进，表明使优化意图显式化对于实现全局协调、语义合理和高性能的 IR 优化至关重要。